Lab 16 – Database Design and Queries

Name: Roger A Raju
Roll.no: 2503A52370
Batch: 13

Task Description #1 (Design Database Schema for Student Management System)

Prompt:

Generate SQL CREATE TABLE statements for a Student Management System with three tables: Students, Courses, and Enrollments. Include appropriate data types, primary keys, foreign keys, and constraints. Ensure the database design follows 1NF, 2NF, and 3NF. Keep the SQL code simple and compatible with MySQL.

Code:

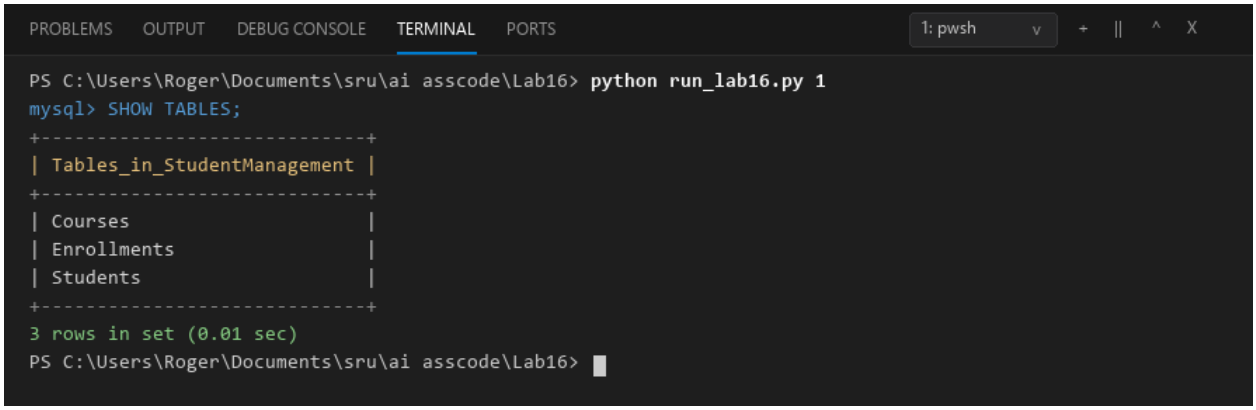
```
CREATE DATABASE StudentManagement;
USE StudentManagement;

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT
);

CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Output:



Justification:

Created three tables: Students, Courses, and Enrollments using SQL. Primary keys uniquely identify records, while foreign keys establish relationships between tables. The database follows 1NF, 2NF, and 3NF to reduce data redundancy and maintain data integrity.

Task Description #2 (Insert Sample Records)

Prompt:

Generate SQL INSERT statements to insert at least 3 students, 4 courses, and 5 enrollment records into the Students, Courses, and Enrollments tables. Also generate SELECT queries to verify the inserted records.

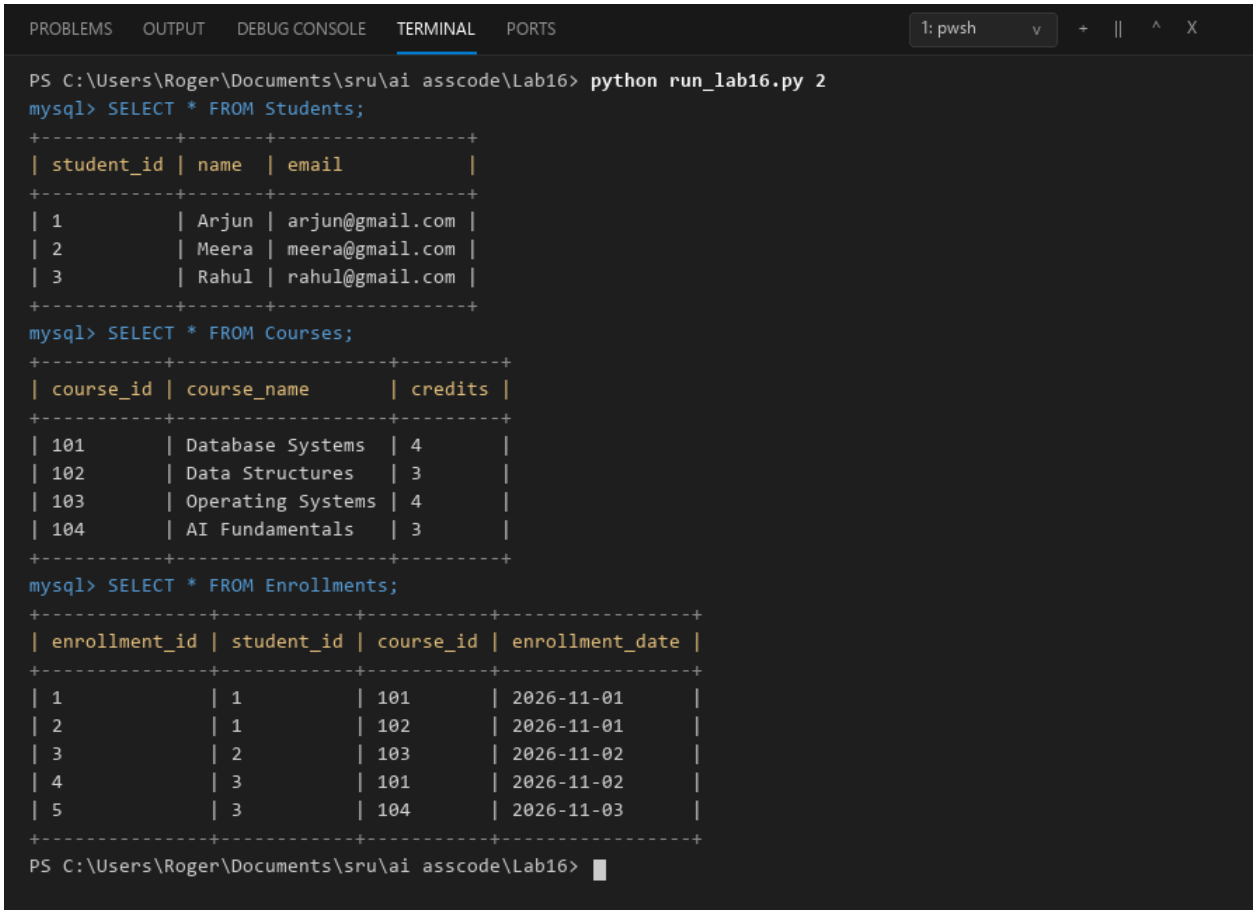
Code:

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES
(1, 1, 101, '2026-11-01'),
(2, 1, 102, '2026-11-01'),
(3, 2, 103, '2026-11-02'),
(4, 3, 101, '2026-11-02'),
(5, 3, 104, '2026-11-03');
```

Output:



Justification:

Inserted sample records into Students, Courses, and Enrollments tables using INSERT statements. Used SELECT queries to verify the inserted records successfully.

Task Description #3 (Perform CRUD Operations)

Prompt:

Generate SQL queries to perform CRUD operations on the Student Management Database. Retrieve all courses, update a student's email, delete an enrollment record, and verify the changes using SELECT statements.

Code:

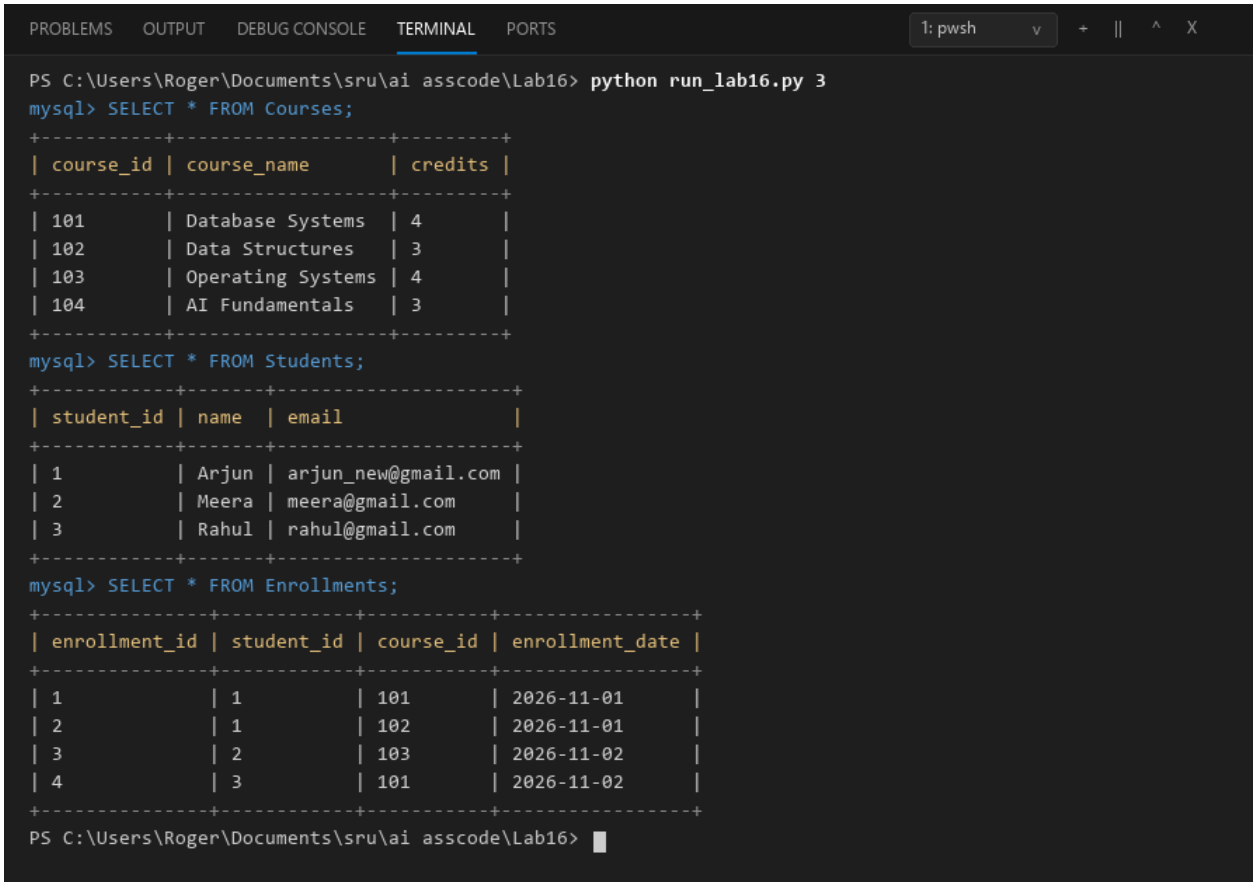
```
SELECT * FROM Courses;

UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;

DELETE FROM Enrollments
WHERE enrollment_id = 5;

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output:



Justification:

Performed CRUD operations using SELECT, UPDATE, and DELETE statements. Verified the updated email and deleted enrollment record using SELECT queries.

Task Description #4 (Execute Aggregate Queries)

Prompt:

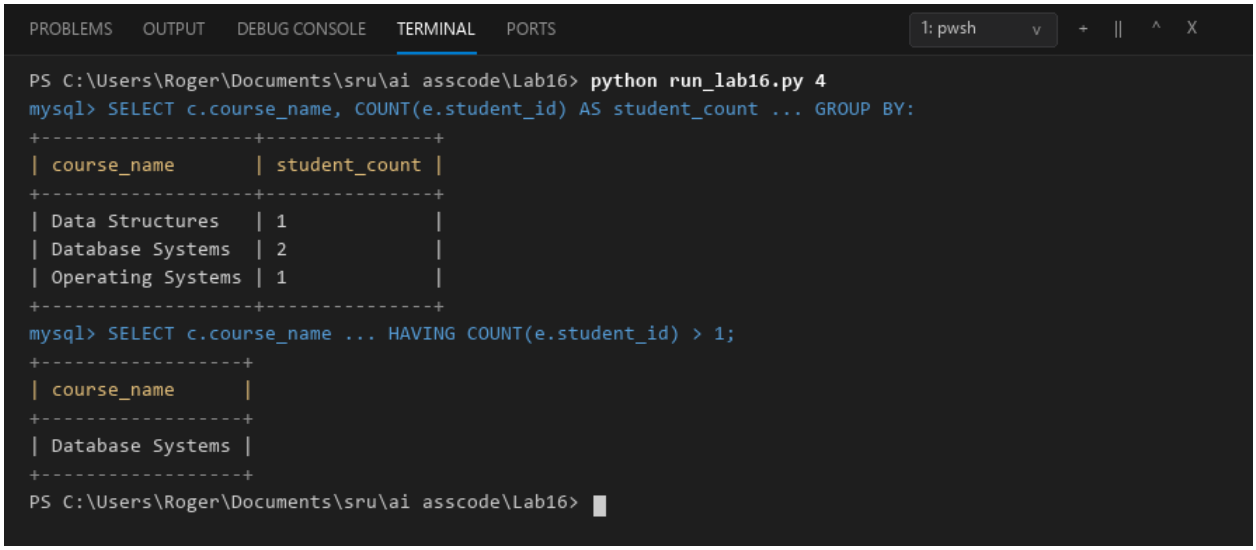
Generate SQL aggregate queries to count the number of students enrolled in each course and display courses having more than one student enrolled using GROUP BY, COUNT(), and HAVING clauses.

Code:

```
SELECT c.course_name,
       COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM Courses c
JOIN Enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Output:



Justification:

Used aggregate functions COUNT(), GROUP BY, and HAVING to count students enrolled in each course and identify courses with more than one student.

Task Description #5 (Query Optimization)

Prompt:

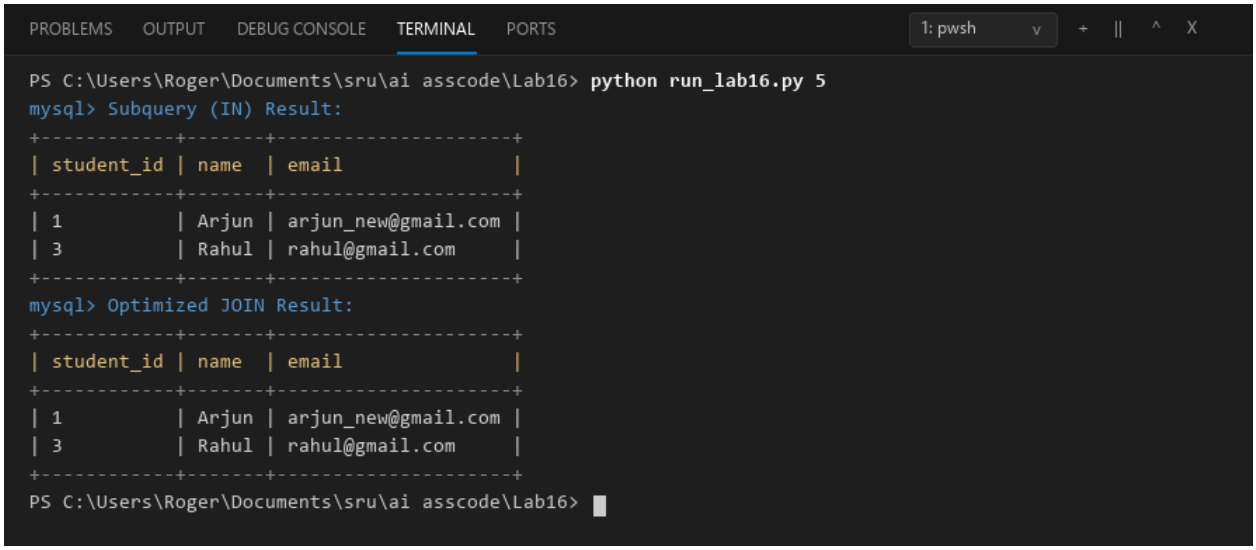
Analyze the given SQL subquery and rewrite it using JOINs for better query performance. Ensure both queries return the same results.

Code:

```
SELECT * FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 101
);

SELECT s.*
FROM Students s
JOIN Enrollments e
ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Output:



Justification:

Rewrote the original subquery using JOIN to retrieve students enrolled in a specific course. Compared both queries to verify that they produce the same results and improve query efficiency.